

LeetCode 1819: Number of Different Subsequences GCDs

Akuna Capital

Amazon

Problem Reference: Q.20 | Difficulty: Hard

Problem Description & Context

Given an array of positive integers `nums`, return the number of **distinct Greatest Common Divisors (GCDs)** among all non-empty subsequences of `nums`.

Example Walkthrough: Let the input array be `[6, 10, 3]`.

Subsequence	Computed GCD
[6]	6
[10]	10
[3]	3
[6, 10]	2
[6, 3]	3
[10, 3]	1
[6, 10, 3]	1

Distinct GCDs found: {1, 2, 3, 6, 10} → **Total Count = 5**

1. Brute Force Approach

- **Step 1:** Generate every possible subsequence of the array.
- **Step 2:** Compute the GCD for each generated subsequence.
- **Step 3:** Insert the resulting GCDs into a hash set to filter unique values.

Complexity Analysis: Since there are 2^n possible subsequences, this approach results in an exponential time complexity (**Very Complex TC**), which will easily lead to a Time Limit Exceeded (TLE) error for larger inputs.

2. Optimal Thinking (Inverse Search Strategy)

Instead of searching over all possible subsequences, we change our perspective and **search over all possible GCD values**.

We ask the fundamental question: *"Can a specific GCD value g occur?"*

For a subsequence to have a greatest common divisor equal to g , **every number chosen in that subsequence must be a multiple of g** . Therefore, if we collect all numbers in the array that are divisible by g , their cumulative GCD must equal exactly g .

Case Study 1: Testing if $g = 3$ can occur in $[6, 9, 15]$.

- All numbers are divisible by 3.
- $\text{gcd}(6, 9) = 3$
- $\text{gcd}(6, 9, 15) = 3$
- Since $\text{curr_gcd} == g$, $g = 3$ is **Achievable**.

Case Study 2: Testing if $g = 4$ can occur in $[8, 16, 24]$.

- All numbers are divisible by 4.
- $\text{gcd}(8, 16, 24) = 8$
- Since $\text{curr_gcd}(8) > g(4)$, every subset will have a GCD that is a multiple of 8, and we can **never** achieve exactly 4. Hence, $g = 4$ is **Not Achievable**.

| 3. Optimized Algorithm

To avoid searching the entire array multiple times, we use a **boolean look-up array / lookup structure** where $\text{present}[g] == \text{true}$ if g exists in the array.

- Iterate g from 1 to max_val .
- For each g , check its multiples: $g, 2g, 3g, \dots, xg$.
- If a multiple exists in the array, update: $\text{curr_gcd} = \text{gcd}(\text{curr_gcd}, \text{multiple})$.
- If at any point $\text{curr_gcd} == g$, increment the answer count and move to the next candidate.

| 4. Implementation Code

C++ Solution:

```
#include <vector>
#include <algorithm>
#include <numeric>

class Solution {
public:
    int countDifferentSubsequenceGCDs(std::vector<int>& nums) {
        int max_val = *std::max_element(nums.begin(), nums.end());
        std::vector<bool> present(max_val + 1, false);

        for (int num : nums) {
            present[num] = true;
        }
    }
};
```

```

    int ans = 0;

    // Iterate over all possible GCD values
    for (int g = 1; g <= max_val; ++g) {
        int curr_gcd = 0;

        // Look at multiples of g: g, 2g, 3g...
        for (int multiple = g; multiple <= max_val; multiple += g) {
            if (present[multiple]) {
                curr_gcd = std::gcd(curr_gcd, multiple);
                if (curr_gcd == g) {
                    ans++;
                    break; // Found the minimum subset forming GCD g
                }
            }
        }
    }
    return ans;
}
};

```

Python Solution:

```

import math

class Solution:
    def countDifferentSubsequenceGCDs(self, nums: list[int]) -> int:
        max_val = max(nums)
        present = [False] * (max_val + 1)

        for num in nums:
            present[num] = True

        ans = 0

        # Iterate over all possible GCD values
        for g in range(1, max_val + 1):
            curr_gcd = 0

            # Look at multiples of g: g, 2g, 3g...
            for multiple in range(g, max_val + 1, g):
                if present[multiple]:
                    curr_gcd = math.gcd(curr_gcd, multiple)
                    if curr_gcd == g:
                        ans += 1
                        break

        return ans

```

